Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

Control de Potencia Bluetooth

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 6 de marzo de 2026

Índice

Introducción 3

Descripción de la Esp32 4

Descripción de la NRF24L01 4

Diferenciación entre las NRF24L01 con y sin PA+LNA con y sin antena 5

Circuitos 6

1. Transmisor 6
   a) Diseño 6
   b) Código 7
2. Receptor 9
   c) Diseño 9
   d) Código 10

Conclusiones 14

Enlace a GitHub 14

Introducción

En este proyecto se implementó la comunicación inalámbrica entre dos microcontroladores ESP32 utilizando módulos NRF24L01. El objetivo fue establecer un enlace confiable de transmisión y recepción de datos en la banda de 2.4 GHz, configurando parámetros como canal, potencia y velocidad de transmisión para optimizar estabilidad y alcance. Se desarrollaron dos programas: uno para el transmisor, encargado de enviar mensajes periódicos, y otro para el receptor, que procesa la información recibida y la muestra en el puerto serial. Esta práctica permitió comprender el funcionamiento de los módulos NRF24L01 y su integración con el ESP32 mediante la librería RF24 y la interfaz SPI.

Descripción de la Esp32

La ESP32 es un microcontrolador versátil con Wi-Fi y Bluetooth integrados:

Características principales:

Procesador dual-core Tensilica LX6 a 240 MHz.

Conectividad Wi-Fi (802.11 b/g/n) y Bluetooth/BLE integrada.

Amplia cantidad de GPIOs para sensores, actuadores y periféricos.

Soporte para protocolos como SPI, I2C, UART, PWM y ADC.

Ventajas: Ideal para proyectos IoT, bajo costo, gran comunidad de soporte y capacidad de ejecutar tareas complejas con conectividad inalámbrica.

Descripción de la NRF24L01

NRF24L01 es un transceptor de 2.4 GHz diseñado para comunicaciones de bajo costo y bajo consumo.

Tipo: Transceptor inalámbrico de 2.4 GHz.

Funciones:

Comunicación bidireccional (envío y recepción).

Velocidades configurables: 250 kbps, 1 Mbps y 2 Mbps.

Bajo consumo de energía, adecuado para dispositivos portátiles.

Uso típico: Comunicación entre microcontroladores (Arduino, ESP32, STM32) en proyectos de domótica, sensores inalámbricos y control remoto.

Diferenciación entre las NRF24L01 con y sin PA+LNA con y sin antena

La versión con PA+LNA y antena externa ofrece mayor alcance y estabilidad que la versión estándar.

Versión	PA (Power Amplifier)	LNA (Low Noise Amplifier)	Antena	Alcance aproximado	Consumo
NRF24L01 estándar (sin PA/LNA, sin antena)	No	No	PCB integrada	10–30 m en interiores	Muy bajo
NRF24L01 con antena externa (sin PA/LNA)	No	No	Antena SMA	50–100 m	Bajo
NRF24L01 con PA+LNA y antena externa	Sí	Sí	Antena SMA	500–1000 m en condiciones óptimas	Más alto

PA (Power Amplifier): Incrementa la potencia de transmisión, extendiendo el alcance.

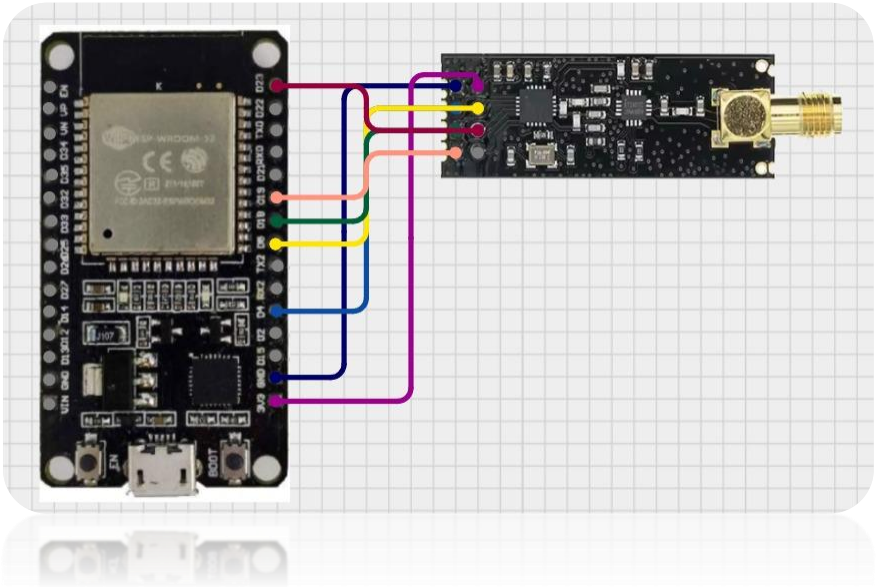
LNA (Low Noise Amplifier): Mejora la sensibilidad de recepción, reduciendo interferencias.

Antena externa: Proporciona mejor ganancia y estabilidad frente a obstáculos.

Circuitos

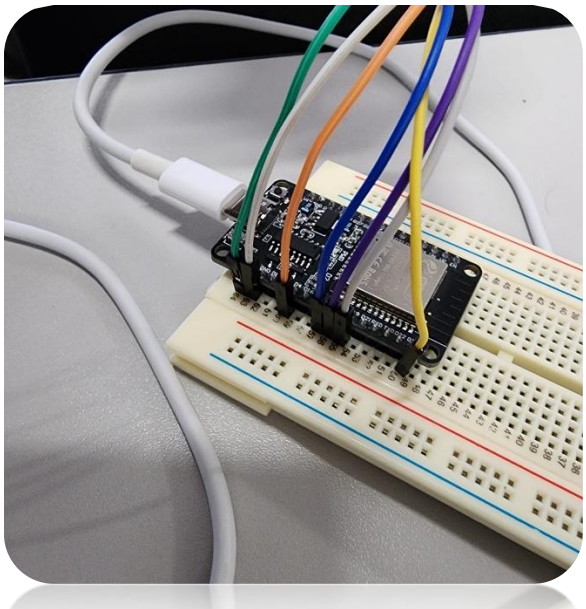
1. Transmisor

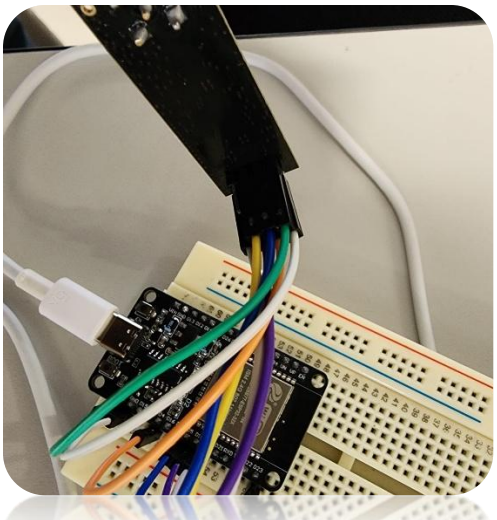
a) Diseño



Sobre una protoboard se han conectado, tanto de la antena como el ESP32, los pines tal como se muestran en la imagen, manteniendo tan solo uno sin conexión.

Tras suministrarle energía, la antena se calentó ligeramente durante la primera prueba, sin embargo, no ocurrió más tras conectarlo nuevamente con el código cargado en el microprocesador.





b) Código

Librerías que permiten la comunicación de los NRF24L01 y facilita su manejo

```
#include <SPI.h>

#include <RF24.h>
```

Aqui se define los pines del ESP-32 para controlar el módulo NRF24L01

```
#define CE_PIN 4

#define CSN_PIN 5
```

Se crea el objeto radio con los pines definidos y se crea una dirección de comunicación que se usara para ambos dispositivos

```
RF24 radio(CE_PIN, CSN_PIN);

const byte direccion[6] = "00001";
```

Se configurará el setup con una comunicación serial en 115200

```
void setup() {
```

```
  Serial.begin(115200);
```

Esta sentencia inicia el módulo NRF24L01

```
  if (!radio.begin()) {
    Serial.println("NRF24 NO detectado");
    while (1);
  }
```

Se configuran valores para la comunicación de los módulos. Se usará el canal 69 en 2.4 GHz, configurado para que use la máxima distancia posible y que use una velocidad de 250KBps para tener una transmisión más confiable y estable.

```
  radio.setChannel(69);
  radio.setPALevel(RF24_PA_MAX);
  radio.setDataRate(RF24_250KBPS);
```

Se abre el canal de transmisión y se active el módulo para que funcione como transmisor en modo no escuchar.

```
 radio.openWritingPipe(direccion);
  radio.stopListening();
  Serial.println("Transmisor listo (ESP32)");}
```

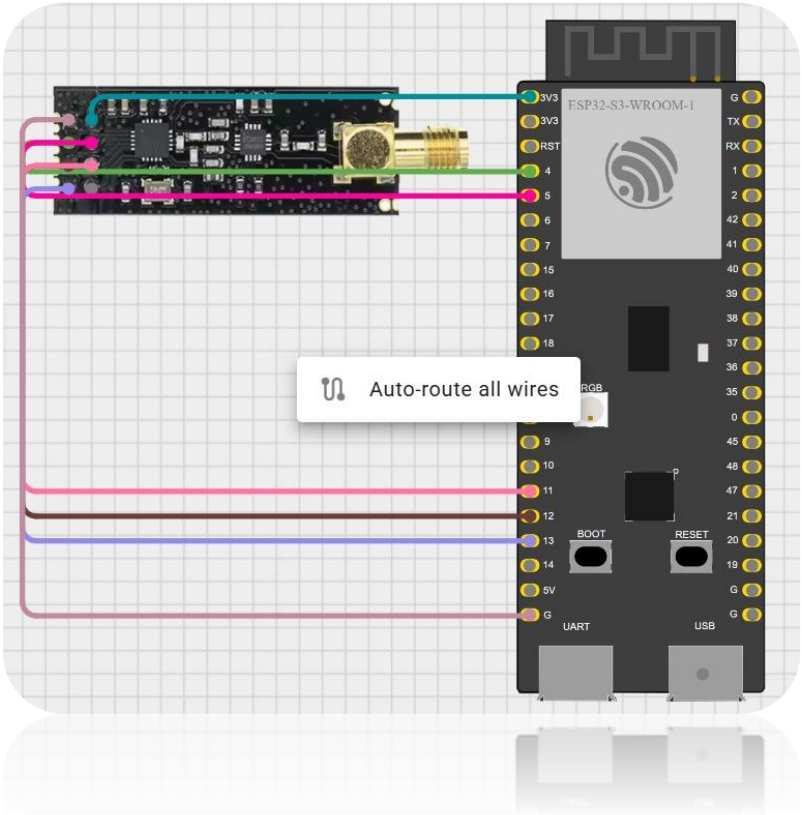
Es el mensaje que se enviara a través de la comunicación entre ambos módulos. Se hace una comprobación si se mandó correctamente el mensaje y notifica sobre su estado teniendo un delay de un segundo entre cada loop.

```
void loop() {
  const char texto[] = "Pago mandado";
  bool ok = radio.write(&texto, sizeof(texto));
```

```
  if (ok) {
    Serial.println("Enviado correctamente");
  } else {
    Serial.println("Error al enviar");
  }
  delay(1000);
}
```

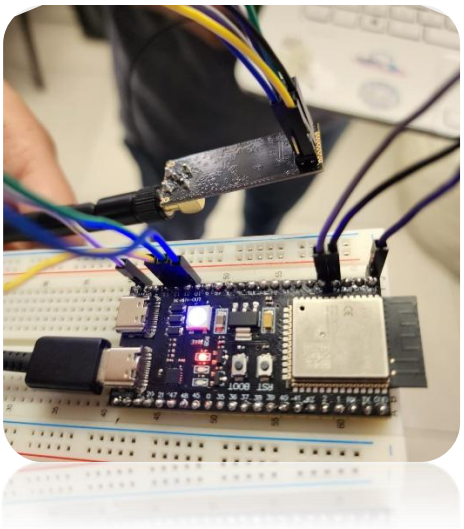
2. Receptor

c) Diseño



En la imagen anterior se podrá observar también la conexión entre un ESP32 algo distinto con una antena similar, nuevamente, se ha dejado el último pin sin conectar y de esta manera está listo para cargarle el código y comprobar el ‘handshake’ entre estos dos ESP32.

Tras proporcionarles energía, la antena no despidió calor ni tuvo mayores problemas.



d) Código

Librerías que permiten la comunicación de los NRF24L01 y facilita su manejo

```
#include <SPI.h>

#include <RF24.h>
```

Aqui se define los pines del ESP-32 para controlar el módulo NRF24L01

```
#define CE_PIN   4

#define CSN_PIN  5

#define SCK_PIN  12

#define MISO_PIN 13

#define MOSI_PIN 11
```

Se crea el objeto radio con los pines definidos y se crea una dirección de comunicación que se usara para ambos dispositivos

```
RF24 radio (CE_PIN, CSN_PIN);

const byte direccion[6] = "00001";
```

Se configurara el setup con una comunicación serial en 115200

```
void setup() {
  Serial.begin(115200);
```

Se inicia la comunicación SPI con los pines que se definieron

```
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN);
```

Esta sentencia inicia el módulo NRF24L01,

```
  if (!radio.begin()) {
    Serial.println("NRF24 NO detectado");
```

Se utiliza el led integrado para representar los distintos estados de la comunicación, en este caso rojo si ocurre algún error de hardware.

```
    #ifdef RGB_BUILTIN
      // Rojo si hay error de hardware
      rgbLedWrite(RGB_BUILTIN, RGB_BRIGHTNESS, 0, 0);
    #endif
    while (1);
  }

  Serial.println("NRF24 detectado");
```

Se configuran valores para la comunicación de los módulos. Se usara el canal 69 en 2.4 GHz, configurado para que use la máxima distancia posible y que use una velocidad de 250KBps para tener una transmisión más confiable y estable.

```
  radio.setChannel(69);
  radio.setPALevel(RF24_PA_MAX);
```

```
  radio.setDataRate(RF24_250KBPS);
```

Abre el canal de comunicación con una dirección definida y activa el modo de escucha

```
  radio.openReadingPipe(0, direccion);
  radio.startListening();
```

El Led en azul indica que está esperando una señal.

```
  #ifdef RGB_BUILTIN
    // Azul tenue para indicar que está esperando señal
    rgbLedWrite(RGB_BUILTIN, 0, 0, 20);
  #endif
}

void loop() {
```

Verifica si hay datos disponibles para leer, si hay, los muestra en el puerto serial.

```
  if (radio.available()) {
    char texto[32] = "";
    radio.read(&texto, sizeof(texto));

    Serial.print("Recibido: ");
    Serial.println(texto);
```

Led de color verde cuando recibe una señal

```
    #ifdef RGB_BUILTIN
      // Destello Verde al recibir cualquier señal
```

```
    rgbLedWrite(RGB_BUILTIN, 0, RGB_BRIGHTNESS, 0);
    delay(200);
    // Volver a azul tenue
    rgbLedWrite(RGB_BUILTIN, 0, 0, 20);
  #endif
 }
}
```

Ambos códigos demostraron funcionalidad al sincronizarlos, asegurando que compartan el mismo canal de transmisión.

Conclusiones

Ramírez Rodríguez Martín de Jesús:

La implementación de la comunicación inalámbrica entre dos ESP32 mediante módulos NRF24L01 permitió comprobar la eficacia de este sistema en la transmisión y recepción de datos a 2.4 GHz. El uso de la librería RF24 con Arduino IDE y la configuración adecuada de parámetros como canal, potencia y velocidad garantizó un enlace estable y confiable. A partir de esta práctica se ha comenzado a comprender de manera práctica cómo integrar hardware y software para establecer conexiones inalámbricas simples, abriendo la posibilidad de aplicar esta tecnología en proyectos más complejos de automatización y control.

Santacruz Rodarte Cristian Alexis:

En conclusión, con la actividad presentada aprendimos los cimientos básicos en la creación de redes con antenas NRF24, la complementación con el equipo que recibía la señal ayudo amostrar ambos lados en la creación de redes Bluetooth, además de la creación de aplicaciones con el fin de controlar medios por la misma.

Pedroza Pérez Emmanuel:

Gracias a esta actividad entendemos el funcionamiento del módulo NRF24 y como es su función básica para mandar señales de un punto a otro, con este parteaguas nos permite indagar más sobre funciones más complejas para programar y obtener dispositivos más complejos que permitan resolver diferentes tipos de problemas cotidianos.

Enlace a GitHub

Este enlace que se muestra a continuación, muestra tanto el repositorio donde se encuentra la documentación misma, los códigos empleados para esta práctica y su video de evidencia:

https://github.com/MartinITAAgs/Tecnolog-as-Inal-mbricas-12pm/tree/main/1_Pr%C3%A1ctica%20NFR24L01%20-%20HandShake